

Задача 3. (10 точки) Да се напише програма, която въвежда от клавиатурата правоъгълна матрица от реални числа с 5 реда и 10 стълба. Програмата да извежда на екрана текста "yes" в случай, че въведената матрицата има поне два различни линейно зависими реда и "no" в противен случай.

Задача 4. (10 точки) Да се напише програма, която въвежда от клавиатурата масив от 5 символни низа, всеки с максимална дължина 50 символа. Програмата да извежда текста "yes" в случай, че от въведените низове има такъв низ s , на който всички останали низове са поднизове и "no" в противен случай.

Забележка: Низът $a_0 a_1 \dots a_k$ е подниз на низа $b_0 b_1 \dots b_m$, ако $k \leq m$ и съществува такова естествено число i , че $0 \leq i \leq m-k$ и $a_0 = b_i, a_1 = b_{i+1}, \dots, a_k = b_{i+k}$. Например, "abc" е подниз на "0abc1", но не е подниз на "a0b1c".

Задача 3. (12 точки) Да се напише функция, която по зададен масив от низове намира най-големия брой низове от масива, които са *анаграми* помежду си. Един низ е анаграма на друг, ако е съставен от същите символи, но в разбъркан ред, като това означава, че за да са анаграми два низа трябва всеки от символите на първия да се среща точно толкова пъти и във втория. Например низовете asdff и fsdaf са анаграми един на друг, докато низовете asdff и aafsd не са.

Примери:

Вход: string gnrts ringst strong spong shpong pongs

Изход: 3

Думите могат да бъдат разделени на следните групи, елементите на които са анаграми помежду си:

string gnrts ringst

strong

shpong

spong pongs

Задача 3. (12 точки) Дума в текст е всяка редица от малки и големи латински букви, цифри или символа '_'. Всички останали символи се считат за препинателни знаци. Да се напише функция, която преобразува текст по следния начин:

- редицата от думи в текста се обръща в обратен ред като първата дума става последна, втората дума става предпоследна и т.н., последната дума става първа.
- редицата от препинателни знаци в текста остава същата. Това означава, че ако има препинателен знак след първата дума на входния текст, то след преобразуването този препинателен знак е след последната дума от входния текст, която дума е първа в изходния текст.
- думите запазват реда на буквите си.
- препинателните знаци, ако са повече от един между две думи, също запазват реда си.

Примери.

Вход: текст 1. Изход: 1 текст.	Входният текст съдържа две думи: "текст" и "1", а препинателните знаци са ' ' и '.'. В изхода думите сменят реда си, а препинателните знаци остават на местата си.
Вход: текст ! Изход: текст !	Текстът "текст !" не се променя тъй като се състои само от една дума.
Вход: 1 ; ; 2 ; 3 ; 4 ; ; 5. Изход: 5 ; ; 4 ; 3 ; 2 ; ; 1.	Входният текст съдържа пет думи: "1", "2", "3", "4" и "5" и препинателните знаци: ';' и '.'.

Задача 4. (10 т.) *Задачата да се реши на езика C++ или Java. В началото на вашето решение посочете кой език сте избрали.*

Дадена е квадратна матрица от цели числа с размери 10×10 , която описва лабиринт. Стойност 0 в дадена клетка означава „стена“, а стойност 1 означава „проходима клетка“. Даден е символен низ, съдържащ само буквите E, W, N и S, които указват едностъпкови придвижвания в съответните географски посоки: N – нагоре, E – надясно, S – надолу, W – наляво.

Да се напише функция `walk`, която получава матрица и символен низ от вида, определен по-горе и проверява дали символният низ задава валиден път започващ от някоя проходима клетка на лабиринта, състоящ се само от проходими клетки и завършващ в долния десен ъгъл на лабиринта. Функцията да връща булева стойност – *истина*, ако такава клетка има и даденият низ задава валиден път и *лъжа* в противен случай.

11.9.2014 г. СУ-ФМИ

Държавен изпит
за ОКС *Бакалавър*

**Компютърни
науки**

ф.н.

лист
5/13

Задача 4. (10 т.) *Задачата да се реши на езика C++ или Java. В началото на вашето решение посочете кой език сте избрали.*

Дадени са координатите на N -точки, които са записани в масивите `float x[N], y[N]` по следния начин: координатите на i -тата точка са $(x[i], y[i])$.

Напишете функция `square`, която получава като аргументи броя на точките N и два масива X и Y съдържащи координатите им и извежда на екрана координатите на центъра и страната на най-малкия квадрат със страни успоредни на координатните оси, който обхваща всички дадени точки (всички дадени точки са във вътрешността му или на страните му).

Ако решавате задачата на Java, достатъчно е да напишете статична функция, която решава задачата.

Задача 1 (10 точки). Следната задача да се реши на един от езиците за програмиране C++ или Java. Дадена е `float` матрица `img` с размери $M \leq 10$ реда и $N \leq 10$ стълба. Напишете функция `subsample`, която получава като аргументи `M`, `N` и `img` и извежда на екрана матрица `s` с размери $(M+1)/2$ и $(N+1)/2$ (при целочислено деление), всеки елемент `s[i][j]` на която е равен на средно-аритметичното от всички елементи `img[y][x]`, такива че

$$i*2 \leq y \leq i*2+1 \text{ и } j*2 \leq x \leq j*2+1.$$

Например, при матрица `img`, представена таблично по следния начин:

1.0	2.0	3.0
4.5	6.5	7.5

функцията да извежда на екрана:

3.5	5.25
-----	------

Забележка: При избор на Java за език за програмиране е достатъчно да се дефинира статичен метод, който решава задачата.

Задача 1 (10 точки).

А) Следните програмни фрагменти са съответно от булева функция на C++ и статичен булев метод на Java, проверяващи дали в даден масив `a` от цели числа, подредени в нарастващ ред, се съдържа числото `x`. Функцията/методът прилагат алгоритъма за двоично търсене. Липсващите части от фрагментите са обозначени с _____. Попълнете липсващите части така, че функцията или съответно методът да са коректно дефинирани спрямо това описание. Решете задачата за един от двата езика по избор!

```
(C++) bool member (int x, int a[], int size){
    if (size == 0) return false;
    return a[size/2] == x ____
        (a[size/2] < x && member (____)) ____
        (____);
}

(Java) static boolean member (int x, int[] a){
    if (a.length == 0) return false;
    return a[a.length/2] == x ____
        (a[a.length/2] < x && member (____)) ____
        (____);
}
```

Б) (C++ и Java) Нека е дефиниран масив `nums`, в който се съдържат `N` целочислени стойности. Попълнете празните полета, за да бъде коректна програмната реализация на алгоритъма за сортиране във възходящ ред чрез пряка селекция (selection sort).

```
for (int i = 0; i < ____; i++) {
    int min = i;
    for (int j = ____; j < ____; j++) {
        if(nums[____] < nums[____]) ____ = ____;
    }
    if(min != ____) {
        int x = nums[i];
        nums[____] = ____;
        ____;
    }
}
```

В) Каква ще бъде стойността на променливата `result` след изпълнение на следния програмен фрагмент на C++/Java:

```
int a = 0; int b = 15; int result = -1;
if (b < 10 && b / a < 10) result = 0;
else result = 1;
```

а) -1 б) 0 в) 1 г) грешка при компилация делене на нула д) грешка при изпълнение делене на нула

Г) Каква е стойността на израза `6 | 11` на езиците C++ и Java:

а) 15 б) 13 в) 10 г) 11

Задача 1. Дадена е квадратна матрица от дробни числа с размерност 11×11 . Казваме, че два елемента на матрицата са **симетрични**, ако те са разположени симетрично относно (1) главния диагонал, (2) вторичния диагонал, или (3) пресечната точка на двата диагонала на матрицата. На всеки елемент от матрицата съответстват най-много три симетрични елемента. Съвкупност, състояща се от елемент на матрицата заедно с всички негови симетрични елементи, наричаме **симетрична група**.

Пример: В матрицата с размерност 3×3

$$\begin{pmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{pmatrix}$$

$\{a_{01}, a_{10}, a_{21}, a_{12}\}$ и $\{a_{00}, a_{22}\}$ са симетрични групи.

Да се състави функция, която променя матрицата така, че **всеки** елемент a_{ij} , $0 \leq i, j \leq 10$ от матрицата се заменя със средно-аритметичната стойност на елементите в неговата симетрична група.

Задача 1. Задачата да се реши с използване на език за процедурно или обектно-ориентирано програмиране (C, C++ или Java).

Да се състави функция, която приема като параметър низ с произволна дължина и връща като резултат позициите на двойката **еднакви** символи, които са максимално отдалечени един от друг.

Ако в низа съществуват няколко двойки максимално отдалечени символи, функцията да връща позициите на най-ляво разположената двойка. Счита се, че номерата на позициите започват от 0.

Пример:

В символния низ "this is just a simple example" най-ляво и най-дясно разположените символи ' ' (*интервали*), са на позиции съответно 4 и 21, намират се на разстояние 17 символа един от друг и няма друга двойка еднакви символи, които са на по-голямо разстояние един от друг.

СОФИЙСКИ УНИВЕРСИТЕТ
“СВ. КЛИМЕНТ ОХРИДСКИ”



ФАКУЛТЕТ ПО МАТЕМАТИКА И
ИНФОРМАТИКА

Д Ъ Р Ж А В Е Н И З П И Т
ЗА ПОЛУЧАВАНЕ НА ОКС “БАКАЛАВЪР” ПО Компютърни науки

ЧАСТ I (ПРАКТИЧЕСКИ ЗАДАЧИ)

11. 07. 2017 г.

Време за работа – 3 часа

Драги абсолвенти, спазвайте стриктно следните указания:

- ☐ Пишете само на предоставените ви листове без да ги разкопчавате
- ☐ Попълнете горе вдясно **ФАКУЛТЕТНИЯ СИ НОМЕР В ПОЛЕТО НА ВСЯКА НЕЧЕТНА СТРАНИЦА**
- ☐ Решението на всяка задача се разполага в предвиденият за това лист
- ☐ При необходимост пренасяте решението на подпечатан нов лист, предоставян от квесторите
- ☐ Не се допуска използването на персонални електронни устройства.

И з п и т н а т а к о м и с и я в и п о ж е л а в а у с п е ш н а р а б о т а .

Задача 1. (30 min)

Горски терен е представен с помощта на мрежа или двумерен масив с $m \times n$ ($m, n \in [0; 100]$) области (елементи). Елементите на двумерния масив са символи, които имат следното значение:

- **R** – река;
- **S** – скала;
- цифри от **1** до **4**, които означават гъстота на гората.

Теренът се променя на всеки 10 години. Реката и скалите остават постоянни, докато гъстотата на горските площи се променя по следните правила:

- **1** преминава в **2**, **2** в **3**, а **3** в **4**.
- **4** преминава в **3**, ако в съседство има поне 3 области с гъстота **4**, в противен случай остава **4**.

Съседни на дадена област (елемент) от масива са тези области, индексите на които се различават най-много с 1 т.е. всяка област има най-много 8 съседни области.

Напишете функция, която по подаден терен намира вида му след 100 години.
Демонстрирайте използването на функцията в кратка програма.

Примерен вид на терен:

```
R R 1 1 2 2
1 R R R 1 2
S 1 R R 2 3
4 4 S S R R
```

СОФИЙСКИ УНИВЕРСИТЕТ
“СВ. КЛИМЕНТ ОХРИДСКИ”



ФАКУЛТЕТ ПО МАТЕМАТИКА И
ИНФОРМАТИКА

Д Ъ Р Ж А В Е Н И З П И Т
ЗА ПОЛУЧАВАНЕ НА ОКС “БАКАЛАВЪР” ПО Компютърни науки

ЧАСТ I (ПРАКТИЧЕСКИ ЗАДАЧИ)

09. 09. 2017 г.

Време за работа – 3 часа

Драги абсолвенти, спазвайте стриктно следните указания:

- ☐ Пишете само на предоставените ви листове без да ги разкопчавате
- ☐ Попълнете горе вдясно **ФАКУЛТЕТНИЯ СИ НОМЕР В ПОЛЕТО НА ВСЯКА НЕЧЕТНА СТРАНИЦА**
- ☐ Решението на всяка задача се разполага в предвиденият за това лист
- ☐ При необходимост пренасяте решението на подпечатан нов лист, предоставян от квесторите
- ☐ Не се допуска използването на персонални електронни устройства.

И з п и т н а т а к о м и с и я в и п о ж е л а в а у с п е ш н а р а б о т а .

Задача 1. (35 min)

Горски терен е представен с помощта на мрежа или двумерен масив с $m \times n$ ($m, n \in [0; 100]$) области (елементи). Елементите на двумерния масив са символи, които имат следното значение:

- **R** – река;
- **S** – скала;
- цифри от **1** до **4**, които означават гъстота на гората.

Съседни на дадена област (елемент) от масива са тези области, с които тя има обща стена. Две съседни области са свързани, ако в тях има записана една и съща стойност. Множество от области образува площадка, ако между всеки две области от множеството е възможно да се осъществи придвижване, като се минава само през свързани области, и това множество е максималното по включване с това свойство.

Напишете функция, която намира площта на най-голямата непрекъсната площадка с гъстота на гората 4. Площта на една област (елемент) от масива е единица мярка. Демонстрирайте използването на функцията в кратка програма. Използвайте езиците Java, C или C++.

Примерен вид на терен:

```
R R 1 1 2 2
1 R R R 1 2
S 1 R R 2 3
4 4 S S R R
```

Задача 1. *Задачата да се реши на C/C++.*

Да се дефинира функция **sortLex**, която получава като аргументи положителното число **n** и масив **a**, съдържащ **n** на брой цели неотрицателни числа, и ги сортира във възходящ ред относно лексикографската наредба (например, 123 е преди 9 по лексикографската наредба). Да се напише кратка програма, в която да се демонстрира използването на функцията **sortLex**.

За реализацията на функцията **sortLex** не е позволено използването на стандартни библиотечни функции.

Пример:

При подаден масив {13,14,7,2018,9,0}, след изпълнение на функцията **sortLex** масивът ще бъде подреден по следния начин: {0,13,14,2018,7,9}.

Задача 1. Задачата да се реши на един от езиците C или C++.

Професор X забравя постоянно паролата за своя таен сейф, която представлява последователност от цели числа. За да не е нужно да я помни, той скрива подсказки в кабинета си. На всеки ред от библиотеката му са подредени книги, най-много 20 реда книги, с до 30 книги на ред. Книгите на някои от редовете са подредени по азбучен ред на заглавията си, всяко от които е до 100 символа.

Паролата за сейфа на професор X се определя от числата, които задават последователните дължини на думите в заглавията на книгите, разположени точно в средата на редовете, в които книгите са подредени в азбучен ред. Ако на реда има четен брой книги, за паролата се използва книгата, намираща се по-близо до началото на реда. Думите в заглавията на книгите са разделени от точно един интервал. Дължините на думите формират паролата в реда, в който се срещат, от най-горния към най-долния ред на библиотеката.

Библиотеката на X може да се представи като двумерен масив **a** от низове с **m** реда по **n** низа всеки, представящи заглавията на книгите. Да се дефинира функция **revealPassword**, която по подадени **a**, **m** и **n**, извежда на стандартния изход паролата на професор X като последователност от числа, разделени с по един интервал. Да се демонстрира употребата на функцията **revealPassword** в кратка програма.

Пример:

Нека библиотеката на професор X се състои от 3 реда с по 3 книги всеки както следва:

Алгебра	<u>Аналитична геометрия</u>	Математически анализ
Увод в програмирането	Обектно-ориентирано програмиране	Структури от данни и програмиране
Бази от данни	<u>Изкуствен интелект</u>	Функционално програмиране

Тогава паролата на професор X е поредицата от числа 10 9 9 8, получена от дължините на подчертаните думи. Думите “Обектно-ориентирано програмиране” не участват в паролата, защото книгите на втория ред не са подредени по азбучен ред на заглавията си.

Задача 1. Задачата да се реши на езика C++.

Даден е двумерен масив с размер 6 на 6 от символи — малки и главни латински букви и цифри. Две клетки в него ще наричаме “съседни”, ако имат обща стена (т.е. всяка клетка е съседна с най-много четири други, намиращи се под, над, вляво и вдясно от нея). Път с дължина N ще наричаме редица $a_0, a_1, \dots, a_N$ — от клетки, за която:

1. за всяко $0 \leq i < N - 1$ е изпълнено, че a_i и a_{i+1} са съседни;
2. никоя от клетките не се среща повече от веднъж (т.е. няма цикли).

Да се попълнят празните места в кода на дадените по-долу функция `contains` и помощната ѝ функция `walk`. Функцията `contains` получава два аргумента — масив `arr` от дадения тип `char[6][6]` и символен низ `str`. Тя трябва да връща истина тогава и само тогава, когато в `arr` съществува път, чиито клетки образуват точно съдържанието на низа `str` (вижте примера по-долу). За определеност считаме, че функцията трябва да връща истина за празния низ.

Пример: За дадения по-долу двумерен масив `contains` трябва да върне истина, ако ѝ бъдат подадени низовете `"abcdefgh"`, `"A123B123C"` или `""`. За улеснение, за да може да ги видите по-лесно, те са маркирани в сиво.

y	u	f	a	b	c
G	o	p	g	B	1
c	b	a	h	3	2
d	k	j	i	2	3
e	f	Q	N	1	C
h	g	h	M	A	r

Кодът на двете функции е даден на следващия лист:



```
bool contains(char arr[6][6], const char* str)
{
```

```
    for (int row = 0; row < ____; ____)  
        for (int col = 0; col < ____; ____)  
            if (walk(arr, row, col, str))  
                return ____;  
    return ____;  
}  
  
bool walk(char arr[6][6], int row, int col, const char* str)  
{  
    if (*str == '\\0')  
        return ____;  
  
    if (row < 0 || col < 0 || row >= 6 || col >= 6)  
        return ____;  
  
    if (arr[row][col] != *str)  
        return ____;  
  
    arr[row][col] *= -1;  
  
    bool result =  
        walk(arr, row + __, col, str + 1) ||  
        walk(arr, ____, ____, str + 1) ||  
        walk(arr, ____, ____, str + 1) ||  
        walk(arr, ____, ____, str + 1);  
  
    arr[row][col] ____;  
  
    return result;  
}
```

Задача 1. Задачата да се реши на езика C/C++.

Да се попълнят празните места в кода на функциите така, че те да отговарят на описанието си.

```
// А) find използва алгоритъма за двоично търсене (binary search), за да
// провери дали value се съдържа в масива arr, състоящ се от size елемента.
// Функцията връща true ако това е така и false в противен случай.

bool find(int value, int* arr, size_t size)
{
    if (size == 0) return _____;

    size_t mid = size / 2;
    if (value == arr[mid]) return _____;

    if (value < arr[mid])
        return find(_____, _____, _____);
    else
        return find(_____, arr + _____, _____);
}

// Б) fold_left изпълнява ляво свиване (left fold) върху масива arr, съдържащ size елемента,
// прилагайки операцията op. Началната стойност е nil.
// Функцията връща стойността op(...op(op(nil, a[0]), a[1]), ..., a[size-1]).

template <typename ReturnType, typename InputType, typename OpType>
ReturnType fold_left(InputType* arr, size_t size, OpType op, ReturnType nil)
{
    _____ result = _____;

    for (size_t i = 0; i < _____; ++i)
        result = op(_____, _____);

    return result;
}

int op(char Digit, int Result)
{
    return (_____ * 10) + (_____ - '0');
}

// Преобразува символен низ от десетични цифри до величина от тип int
int str_to_int(const char * str)
{
    return (str == nullptr) ? 0 : fold_left(str, _____, op, _____);
}
```

Задача 1. Да се дефинира рекурсивна функция на езика C++
`void print_backwards(const char* begin, const char* end).`

Параметрите `begin` и `end` са указатели към елементи на буфер от символи, като `end ≥ begin`. Символен низ с начало `begin` и край `end` наричаме последователността от символи, започваща със символа, намиращ се на адрес `begin` и завършваща със символа на адрес `end-1`, включително, дори и ако някой от тези символи е с код 0. Дума в такъв символен низ наричаме всяка непразна подпоследователност от стандартни символи (с код > 32), ограничена от двете страни или от някой от краищата на низа, или от специален символ (с код ≤ 32).

Функцията `print_backwards` да извежда на стандартния изход всички думи в низа с начало `begin` и край `end` в ред, обратен на срещането им в низа, и разделени с точно един интервал.

Да се дефинира функция
`void print_backwards(const char* string),`
която прилага рекурсивната функция `print_backwards` върху символния низ `string`.

Да се демонстрира извикването на последната функция в кратка програма.

Пример: Извикването `print_backwards("I\tneed a break!");` извежда низа "break! a need I".

Задача 1. а) Имената на служители в дадена компания и на техните преки ръководители можем да представим с двумерен масив `const char* leaders[][2]`.

Например:

Служител	Ръководител
Иван Иванов	Мария Иванова
Мария Иванова	Иван Драганов
Иван Драганов	Стоян Петров

Казваме, че служителят *A* е ръководител на служителя *B*, ако *A* е пряк ръководител на *B* или е пряк ръководител на ръководител на *B*. Да се дефинира рекурсивна функция:

```
bool is_subordinate (const char* employee,  
                    const char* manager,  
                    const char* leaders[][2],  
                    size_t n),
```

която проверява дали служителят с име `employee` е подчинен на служителя с име `manager` в компанията, описана с масива `leaders` с `n` реда.

б) Да се дефинира функция

```
const char* the_big_boss(const char* leaders[][2], size_t n),
```

намираща името на служителя, който се намира най-високо в йерархията на компанията, описана с масива `leaders` с `n` реда.

Приемаме, че йерархията от служителите, описана в `leaders` е коректна, т.е. всеки служител има по точно един пряк ръководител, с изключение на точно един служител, който няма пряк ръководител, и няма двама служители такива, че всеки от тях е ръководител на другия.

Да се демонстрира извикването на функциите с кратка програма.

Задача 1. Графично изображение е представено с матрица от m реда и n колони. Клетките на матрицата, във всяка от които е записана целочислена стойност от 0 до 255, съответстват на пикселите в графичното изображение (формат *grayscale*).

Всяка клетка в матрицата има до 8 съседа — до 4 по диагонал, до два, разположени хоризонтално, и до два – вертикално.

Област в изображението е непрекъснатата последователност от съседни клетки с ненулеви стойности. Черните елементи, представени със стойност 0, се считат за контури на областите. Така, една област се определя от граница от нулеви елементи и границите на матрицата.

А) Дефинирайте функция, която получава като аргумент матрица от посочения вид и извежда на стандартния изход средната яркост на всяка от областите, сортирани в низходящ ред според яркостта. Средна яркост на дадената област се изчислява като средно-аритметично на стойностите на всички клетки (пиксели), образуващи областта. За всяка област изведете координатите на една произволна клетка от нея и средната яркост на областта.

Б) Дефинирайте функция `main`, която илюстрира извикване на функцията, дефинирана към т. А. Не се изисква да се реализира вход на данните.

Задачата да се реализира на езика C/C++. Позволява се използването на функции и класове от стандартната библиотека.

Пример: При входна матрица, $m = 6$, $n = 6$

170	0	0	255	221	0
68	0	17	0	0	68
221	0	238	136	0	255
0	0	85	0	136	238
238	17	0	68	0	255
85	170	0	221	17	0

Програмата ще изведе следните области и средни яркости, сортирани в низходящ ред според яркостта:
(0, 0) 153; (0, 3) 151.11

Задача 1. Решете задачата на езика C++. Решения на друг език носят нула точки. Отговорите на подточки 1А, 1Б, 1В и 1Г трябва да съвпадат с това, което би извела програмата. При несъответствие отговорът се оценява с нула точки. Решението на подточка 1Д трябва да бъде технически издържано (например не бива да изтича памет, трябва да се спазват добрите практики за структуриране на програмата и т.н.). Ако решението съдържа сериозни грешки, то се оценява с нула точки.

1А) (1 точка) Какъв ще бъде изходът от изпълнението на следния фрагмент:

```
int calc(int a[5])
{
    int sum = 8;
    for (int i = 0; a[i]; ++i)
        sum += i;
    return sum;
}

int a[7] = {1, 2, 3};
std::cout << calc(a);
```

Отговор: _____

1Б) (1 точка) Какъв ще бъде изходът от изпълнението на следния програмен фрагмент?

```
char text[] = "hello", *p = text;
while (*p) std::cout << ++*p++;
```

Изберете един от следните отговори:

- а) Грешка по време на компилация.
- б) Грешка по време на изпълнение.
- в) ifmmp
- г) el
- д) Безкраен цикъл.

1В) (1 точка) Какво ще изведе следният фрагмент (приемаме, че е част от валидна програма):

```
int *pt;
int a[3] = {4, 19, 13};
pt = &a[1];
pt += 1;
std::cout << *pt << std::endl;
```

Отговор: _____

1Г) (1 точка) В дадените по-долу празни места попълнете какви ще бъдат стойностите на елементите на двата масива А и В след обръщението към функцията f.

```
void f(int * arr1, const int * arr2)
{
    int *p1 = arr1;
    const int *p2 = arr2;

    while(*p2 >= 0)
    {
        *p1++ = *p2++;
    }
}

void main()
{
    int A[4] = {-1, -2, -3, -4};
    int B[4] = {10, 20, 30, -1};

    f(A, B);
}
```

Отговор:

A[0] = ____ A[1] = ____ A[2] = ____ A[3] = ____

B[0] = ____ B[1] = ____ B[2] = ____ B[3] = ____

1Д) (6 точки) Дадени са структура `Point`, описваща точка в декартова координатна система с координати `x` и `y` от тип `float`, и структура `Circle`, описваща окръжност с център `center` от тип `Point` и радиус `r` от тип `float`.

Да се дефинира функция `findRelativePosition`, която определя относителната позиция на две дадени окръжности една спрямо друга. Резултатът от изпълнението на функцията е стойност от изброяения тип:

```
RelativePosition {NO_COMMON_POINTS, TOUCHING,  
                  INTERSECTING, SAME}.
```

със следния смисъл:

- `NO_COMMON_POINTS`: без общи точки
- `TOUCHING`: допиращи се
- `INTERSECTING`: пресичащи се
- `SAME`: съвпадат

Задача 1.

Задачата да се реши на езика C++. В подточките, в които се изисква да се посочи какво ще се изведе, точки се дават само ако отговорът напълно съвпада с това, което извежда съответният код.

1) Дадена е дефиницията:

```
void print(int* arr, int size) {  
    for (int i = 0; i < size; ++i)  
        cout << arr[i] << " ";  
    cout << "\n";  
}
```

Какво ще изведе на стандартния изход даденият по-долу фрагмент? Всеки отделен ред от изхода да се попълни в съответното поле.

```
const int size = 5;  
int arr[] = { 5, 10, -1, 5, 6 };  
for (int i = 0; i < size; ++i) {  
    for (int j = 1; j < size; ++j) {  
        if (arr[j-1] < arr[j]) {  
            swap(arr[j-1], arr[j]);  
        }  
    }  
    print(arr, size);  
}
```

На ред 1 извежда: _____

На ред 2 извежда: _____

На ред 3 извежда: _____

На ред 4 извежда: _____

На ред 5 извежда: _____

2) Какво трябва да се попълни на мястото на коментара в `toUpper` така, че ако на функцията се подаде малка латинска буква, тя да връща съответната ѝ главна, а всички други символи да връща непроменени?

```
char toUpper(char c) {  
    return /* какво трябва да има тук? */ ;  
}
```

A) `(c >= 'a' || c <= 'z') ? (c - 'a' + 'A') : c`

Б) `(c >= 'a' || c <= 'z') ? (c + 'A') : c`

В) `(c >= 'a' || c <= 'z') ? (c - 'a') : c`

Г) `(c >= 'a' && c <= 'z') ? (c - 'a' + 'A') : c1`

Д) `(c >= 'a' && c <= 'z') ? (c + 'A') : c1`

Е) `(c >= 'a' && c <= 'z') ? (c - 'a') : c1`

3) Да се попълнят празните места в дефинициите на функциите така, че `isOdd` да връща истина тогава и само тогава, когато подаденото ѝ число е нечетно, а `isEven` — когато то е четно.

```
bool isEven(unsigned int);  
bool isOdd(unsigned int n) {  
    if (n == 0) return _____;  
    if (n == 1) return _____;  
    return isEven(_____);  
}  
bool isEven(unsigned int n) {  
    if (n == 0) return _____;  
    if (n == 1) return _____;  
    return isOdd(_____);  
}
```

4) Под всеки от дадените по-долу фрагменти да се посочи какво ще изведе той на стандартния изход.

```
for (int i = 0; i < 9; ++i)  
    cout << (1 << i) << " ";
```

```
switch(5 % 2) {  
case 0:  
    std::cout << "0";  
case 1:  
    std::cout << "1";  
default:  
    std::cout << "x";  
}
```

```
for (int i = 0; i < 5; ++i) {  
    if ( ! (i - 3))  
        continue;  
    std::cout << i;  
}
```

```
int arr[] = { 1, 2, 3, 4, 0 };  
*(arr + 2) *= 10;  
for (int* p = arr; *p; p++)  
    std::cout << *p << " ";
```

¹В дадената на изпита тема на тези отговори беше допусната печатна грешка, която тук е отстранена.

Задача 1. Задачата да се реши на езика C++.

1) Нека е дефиниран масивът

```
int arr[] = { 1, 2, 3 };
```

Срещу всеки от изразите да се посочи каква ще бъде неговата оценка.

```
arr[1] == *(arr+2) _____  
arr == &arr[0] _____  
(arr+1) == &arr[1] _____  
*arr == arr[0] _____
```

2) Нека е дадена следната дефиниция:

```
void mystery(const char* str)  
{  
    while (*str && *(str+1)) {  
        std::cout << *str;  
        str += 2;  
    }  
}
```

Да се посочи какво ще изведе на екрана обръщението:

```
mystery("abcdef");
```

3) Нека са дадени следните дефиниции:

```
char s1[] = "Hello";  
char s2[] = "world!";  
char result[80];
```

Да се довърши програмният фрагмент, така че след изпълнението му в `result` да се съхрани коректното представяне на низа "Hello world!". На празните места трябва да се попълнят имената на подходящи стандартни функции за работа с низове.

```
_____(result, s1);  
_____(result, " ");  
_____(result, s2);
```

4) Да се довърши кодът на рекурсивните функции, така че `f` да проверява дали символният низ, сочен от `word`, се съдържа като подниз в `text`. За определеност считаме, че празният низ се съдържа във всеки друг.

```
bool g(const char* text, const char* word)  
{  
    if (!*word) return true;  
    if (!*text) return _____;  
    if (*word != *text) return false;  
    return g(_____, _____);  
}
```

```
bool f(const char* text, const char* word)  
{  
    if (!*word) return _____;  
    if (!*text) return false;  
    return g(_____, _____) ||  
           f(_____, _____);  
}
```

5) Да се посочи какво ще изведе на екрана даденият по-долу фрагмент:

```
char arr[3][3] = { 'a', 'b', 'c',  
                  'd', 'e', 'f',  
                  'g', 'h', 'i' };  
for (int i = 0; i < 3; ++i)  
    std::cout << arr[2-i][i];
```

6) Да се посочи какво ще изведе на екрана даденият по-долу фрагмент:

```
double var = 5 / 2;  
std::cout << var;
```

Задача 1. Задачата да се реши на езика C++.

1) Да се попълнят празните места в кода на функцията `removeWhitespace` така, че тя да премахва от непразния низ `str` всички `whitespace` символи.

```
bool isWhitespace(char c)
{
    return c == ' ' || c == '\t' ||
           c == '\r' || c == '\n';
}

_____ removeWhitespace(char* str)
{
    size_t read=0, write=0;

    while(str[read]){
        if (isWhitespace(_____))
            read_____ ;
        else
            str[write++] = _____ ;
    }

    str[_____] = '\0';

    return str;
}
```

2) Под всеки от фрагментите да се посочи какво ще изведе той на стандартния изход.

```
for (int i = 0; i < 10; ++i) {
    if (i % 2) continue;
    cout << i;
}
```

```
int i = 0x10;
cout << i;
```

```
int a=1,b=2,c=3;
cout << (a ? b : c);
```

```
char str[] = "abc";
char* p = str;
++p;
++*p;
cout << str;
```

3) Да се попълнят празните места в кода на функцията `pass` така, че функцията `bubbleSort` да сортира в нарастващ ред елементите на масива `arr` с размер `size`. Абстрахирайте се от това, че алгоритъмът, разписан по този начин, работи не-ефикасно.

```
void pass(int* arr,
          size_t size,
          bool& swappedAtLeastOnce)
{
    if (size _____ 1)
        return;

    if (arr[0] _____) {
        std::swap(arr[0], arr[1]);
        swappedAtLeastOnce = _____;
    }

    pass(_____,
        _____,
        _____);
}
```

```
void bubbleSort(int* arr, size_t size)
{
    bool swappedAtLeastOnce = false;
    pass(arr, size, swappedAtLeastOnce);
    if (swappedAtLeastOnce)
        bubbleSort(arr, size);
}
```

4) Да се посочи какво ще изведе на стандартния изход следният фрагмент.

```
int x = 2;
int arr[] = {10, 20, 30};
cout << "\nA: " << 5./x;
cout << "\nB: " << (x << 4);
cout << "\nC: " << arr[!x];
cout << "\nD: " << *(arr+x);
cout << "\nE: " << (2 + x++);
```

A: _____
B: _____
C: _____
D: _____
E: _____

Задача 1. Задачата да се реши на езика C++.

- 1) Да се довърши кода на power така, че тя да пресмята повдигането на base на степен power.

```
int power(int base, unsigned int power)
{
    int result = _____;

    while(power > _____) {
        if(power % 2 == _____) {
            _____ = result * base;
        }

        _____ /= 2;
        _____ *= base;
    }

    return result;
}
```

- 2) Да се посочи какво ще изведе на екрана даденият по-долу фрагмент.

```
int a=10, b=010, c=0x10;
cout << a << ", " << b << ", " << c;
```

- 3) Под всеки от фрагментите да се посочи какво ще изведе той на екрана.

```
int var = 0;
while(var++ < 5)
    cout << var;
```

```
int var = 0;
do {
    cout << var;
} while(++var < 5);
```

- 4) Да се посочи какво ще изведе фрагментът. Допускаме, че sizeof(unsigned int) == 4.

```
unsigned int x = 0x11335577;
unsigned char* ptr = (unsigned char*)&x;
++*ptr;
++*(ptr+1);
++ptr[2];
++3[ptr];
cout << hex << "0x" << x;
```

- 5) Нека a и b са променливи от тип char. Да се посочи какъв е проблемът в дадения по-долу фрагмент.

```
char result = a + b;
```

Да се посочи възможен начин, чрез който може да се реши този проблем.

- 6) Нека са дадени следните дефиниции:

```
const size_t size = 3;
char box[size][size] = {
    'a', 'b', 'c',
    'd', 'e', 'f',
    'g', 'h', 'i'
};
```

Да се довършат фрагментите по-долу така, че да извеждат съответните последователности от букви.

```
// Трябва да извежда aei
for (int i = 0; i < size; i++)
    cout << box[_____][_____];
```

```
// Трябва да извежда сег
for (int i = 0; i < size; i++)
    cout << box[_____][size_____];
```

```
// Трябва да извежда gec
for (int i = 0; i < size; i++)
    cout << box[size_____][_____];
```

```
// Трябва да извежда abcfihgd
for(int i = 0; i < size; ++i)
    cout << box[0][i];
for(int i = _____; i < size; _____)
    cout << box[i][size-1];
for(int i = size_____; i >= 0; --i)
    cout << box[size-1][i];
for(int i = size-2; i _____ 0; --i)
    cout << box[i][0];
```

Задача 1. Задачата да се реши на езика C++.

Под всеки от дадените по-долу фрагменти да се посочи какво ще изведе той на стандартния изход. Упътване: ASCII кодовете на символите 'A' и 'a' са съответно 65 и 97.

1)

```
void print(char* p) {  
    if (*(p++)) print(++p);  
    std::cout << *(--p);  
}  
void main() {  
    char str[] = "012345";  
    print(str);  
}
```

Отговор: _____

2)

```
const size_t size = 3;  
int matrix[size][size] = {  
    1, 2, 3,  
    4, 5, 6,  
    7, 8, 9  
};  
std::cout << matrix[1][2] << *matrix[2];  
Отговор: _____  
std::cout << **matrix << *(* (matrix+2)+1);  
Отговор: _____
```

3)

```
char symb = 'A';  
switch(symb) {  
    case 0 : std::cout << "0";  
    case 65 : std::cout << "1";  
    case 'a' : std::cout << "2";  
    default : std::cout << "d";  
}
```

Отговор: _____

4)

```
unsigned x = 8, y = 3;  
while (x > 0) {  
    if (x % y == 0) break;  
    else x -= 1;  
}  
std::cout << x;
```

Отговор: _____

5)

```
unsigned a = 4, b = 8, c = 2;  
std::cout << ((a > c) ? (b > a ? b : a) : c);
```

Отговор: _____

6) Да се попълнят празните места в кода на функцията insert така, че функцията insertSort да сортира в нарастващ ред елементите на масива arr с размер size.

```
void insert(int* arr, int index) {  
    int key = arr[index];  
    int j = _____;  
    while (_____ && _____) {  
        _____;  
        j = j - 1;  
    }  
    arr[j + 1] = key;  
}  
void insertSort(int* arr, int size){  
    for(int i = 1; i < size; i++) {  
        insert(arr, i);  
    }  
}
```

7) Да се попълнят празните места във функцията така, че rotateArray да завърта с k стъпки надясно елементите на масива arr с размер size. Стандартната функция std::reverse(start, end) обръща реда на елементите на масив, зададен чрез указатели start към началото му и end, сочещ **след** последния му елемент.

Пример: При nums = {1, -2, 13, 40, 5} и k = 2 резултатът е {40, 5, 1, -2, 13}.

```
void rotateArray(int* nums,  
    int k, int size) {  
    if (size == 0 || k % size == 0)  
        return;  
    k %= size;  
    std::reverse(nums, nums+size);  
    std::reverse(_____, _____);  
    std::reverse(_____, _____);  
}
```

Задача 1. Задачата да се реши на езика C++.

Под всеки от дадените по-долу фрагменти да се посочи какво ще изведе той на стандартния изход.

1)

```
void print(char* str) {  
    char* ptr = str;  
    while (*ptr != '\0') {  
        *ptr = *ptr+1;  
        ptr++;  
    }  
    std::cout << str;  
}  
void main() {  
    char str[] = "ACE";  
    print(str);  
}
```

Отговор: _____

2)

```
void func(int& x, int* y, int z){  
    int a = 3;  
    int b = 5;  
    x = z - b;  
    *y = 2 * z;  
    z = x ? a + *y : 0;  
}  
void main() {  
    int a = 2; int b = 4;  
    func(b, &a, b);  
    std::cout << a << endl;
```

Отговор: _____

```
    std::cout << b << endl;
```

Отговор: _____

3)

```
int total = 0;  
for (int i = 2; i <= 10; ) {  
    if (i % 3 == 0) {  
        total = total + i;  
    }  
    i += 2;  
}  
std::cout << total << endl;
```

Отговор: _____

4) При какво условие ще спре да се изпълнява цикълът while?

```
while ((a || b) && (c || d)) {...}
```

- а) ако !(a && b) е истина
- б) ако !a || !b || !c || !d е истина
- в) ако (!a && !b) || (!c && !d) е истина
- г) ако (!a || !b) && (!c || !d) е истина
- д) нито един от изброените

Отговор: _____

5) Да се попълнят фрагментите така, че функцията reverseMatrixRows да обръща местата на редовете на mat относно центъра (т.е. първия ред да се размени с последния и т.н.), а функцията reverseMatrixCols да обръща местата на колоните на матрицата mat относно центъра. Нека r и c са съответно брой редове и колони в матрицата.

```
void reverseMatrixRows(  
    int** mat, int r, int c) {  
    for (int i = 0; i < ____; ++i)  
        for (int j = 0; j < c; ++j)  
            swap(mat[i][j], _____);  
}  
  
void reverseMatrixCols(  
    int** mat, int r, int c) {  
    for (int i = 0; i < r; ++i)  
        for (int j = 0; j < ____; ++j)  
            swap(mat[i][j], _____);  
}
```

6) Да се попълнят празните места в рекурсивната функция така, че revStr да обръща елементите на подаден като параметър низ str, които се намират на индекси в интервала [start, end];

```
void revStr(_____ str,  
    unsigned start, unsigned end) {  
    if (_____) {  
        return;  
    }  
    swap(str[start], str[end]);  
    revStr(str, _____, _____);  
}
```